

Script Comparison: Windows 11 Image-Builder Feature Matrix

Comparison of four Windows-11 image-customization tools, plus a fifth reference (Rufus) consulted specifically for its `unattend.xml` generation techniques:

Tool	Location	Role
asl-win11	<code>asl-win11maker.ps1</code> + <code>asl-win11.functions.ps1</code> (repo root)	This project — modular, actively maintained
ntdevlabs	<code>reference/tiny11builder-ntdevlabs/tiny11maker.ps1</code>	Upstream original this project forked from
tiny11-automated (headless)	<code>reference/tiny11-automated/scripts/tiny11maker-headless.ps1</code>	CI/CD-oriented fork of ntdevlabs
winutil	<code>reference/winutil/functions/private/Invoke-WinUtilISOScript.ps1</code>	ISO-customization step embedded in a larger toolkit (ChrisTitusTech WinUtil)
Rufus <i>(reference only)</i>	<code>reference/rufus/src/wue.c</code>	USB-creation tool; not a WIM-servicing pipeline, consulted only for <code>unattend.xml</code> techniques (§6)

All four scripts are git submodules/files kept under `reference/` or this repo purely for comparison — see `CLAUDE.md` for the "reference-only, do not edit" policy.

1. Four-way feature matrix

Feature	asl-win11	ntdevlabs	tiny11-automated (headless)	winutil
Driver injection	✗ none	✗ none	✗ none	✓ <code>Export-WindowsDriver -Online</code> + <code>DISM /Add-Driver /Recurse</code> into <code>install.wim</code> and <code>boot.wim</code> index 2
autounattend.xml handling	Static copy; no XML parsing	Static copy; no XML parsing	Static copy; no XML parsing	✓ Namespace-aware DOM editing (<code>ConvertTo-WinUtilISOAnswerFile</code>) — injects <code>/IMAGE/INDEX</code> , strips empty product keys
Image index injected into XML	✗	✗	✗	✓ (see above)

Feature	asl-win11	ntdevlabs	tiny11-automated (headless)	winutil
Setup script pre-staging from answer file	✗	✗	✗	✓ Parses <code><sg:File></code> extension nodes, writes referenced scripts into the image with correct encoding per extension
Edition enforcement (ei.cfg / PID.txt)	✗	✗	✗	✓ Writes <code>sources\ei.cfg</code> , deletes <code>sources\PID.txt</code>
ISO support\ folder removal	✗	✗	✗	✓
Scheduled tasks removed	5 (see §1a)	5 (identical set)	5 (identical set)	11 (same 5 + 6 more WU-related, see §1a)
App package removal list	49 active prefixes (52 incl. 3 commented out)	~46 prefixes	57 prefixes (largest; adds AI/Recall/Copilot-era packages)	19 prefixes (smallest)
WU/telemetry registry tweaks	Broad (telemetry, sponsored apps, Copilot, Dev Home/Outlook auto-install, new Outlook, Teams) — see <code>asl-win11.functions.ps1</code> <code>Disable-*</code> functions	Same tweaks as asl-win11 (asl-win11 is a modularized fork of this)	Same core set plus <code>DiagTrack</code> / <code>WerSvc</code> / <code>PcaSvc</code> / <code>SysMain</code> service disabling, <code>WindowsAI</code> / Recall keys, <code>DoNotShowFeedbackNotification</code> s	Different tweak set — <code>AU\AUOptions</code> , <code>UseWUServer</code> , <code>WUServer=http://localhost:8080</code> , <code>DODownloadMode=0</code> , BITS/UsSvc/WaaSMedicSvc service disables (see §2 of the original comparison, retained below)
Hardware bypass registry tweaks	LabConfig 5-key bypass + <code>AllowUpgradesWithUnsupportedTPMOrCPU</code> + notification-cache suppression; install.wim only by default (<code>Set-BypassHardwareChecks</code> call is commented out in the pipeline, but <code>Set-BootImageBypassTweaks</code> always applies it to boot.wim)	Identical key set, applied to both install.wim and boot.wim unconditionally	Identical key set, applied to both install.wim and boot.wim unconditionally	No LabConfig-style offline bypass; relies on driver injection + (in the Rufus-style approach) unattend-embedded <code>reg add</code> commands instead
Boot.wim customization	Mounts index 2, applies hardware-bypass tweaks only	Mounts index 2, applies hardware-bypass tweaks only	Mounts index 2, applies hardware-bypass tweaks only	Injects drivers into boot.wim index 2; no hardware-bypass registry tweaks
End-to-end workflow vs. processing step	Full pipeline (ESD convert → mount → strip → tweak → export → ISO)	Full pipeline (monolithic)	Full pipeline (monolithic, CI-oriented)	Processing step only — receives an already-mounted image path/params from the caller

Feature	asl-win11	ntdevlabs	tiny11-automated (headless)	winutil
Logging	<code>Start-Transcript</code> to <code>logs/</code>	<code>Start-Transcript</code> to script root	Custom <code>Write-Log</code> (timestamped, leveled INFO/WARN/ERROR, dual console+file) plus machine-readable <code>tiny11-buildinfo.json</code>	Pluggable <code>-Log</code> scriptblock parameter (caller supplies logging, e.g. WinUtil's own GUI)
Error handling / retry logic	<code>try/catch</code> around a few <code>IsReadOnly</code> /registry ops; no top-level trap	Same minimal pattern; manual re-check-and-remove at final cleanup only	<code>\$ErrorActionPreference='Stop'</code> + <code>Set-StrictMode</code> ; single top-level <code>try/catch</code> with emergency <code>Dismount-WindowsImage -Discard</code> + hive-unload on any failure; explicit <code>exit 0</code> / <code>exit 1</code>	Caller's responsibility (it's a function, not a standalone script)
OneDrive removal	✓ <code>Remove-OneDriveSetup</code> (takeown/icacLS + delete <code>OneDriveSetup.exe</code>)	✓ same technique	✓ same technique + SysWOW64 copy + Run-key removal (HKLM and NTUSER)	✗ not implemented
Edge removal	✓ <code>Remove-Edge</code> (files + optional WebView2 + uninstall registry keys)	✓ same technique, always removes WebView2	✓ same technique + WinSxS <code>microsoft-edge-webview</code> package removal	✗ not implemented
ESD→WIM conversion	✓ prompts for index, <code>Export-WindowsImage -CompressionType Maximum -CheckIntegrity</code>	✓ same	✓ same, non-interactive (index from <code>-INDEX</code> param, validated against ESD image list)	N/A (processing step, assumes WIM already prepared)
DISM cleanup + recovery export	✓ <code>/Cleanup-Image /StartComponentCleanup /ResetBase</code> + <code>/Compress:recovery</code> export pass (install.wim only)	✓ same	✓ same (<code>Optimize-WindowsImage</code> + <code>Dismount-AndExport</code>)	N/A (caller's responsibility)

1a. Scheduled tasks removed, by script

asl-win11 / ntdevlabs / tiny11-automated (identical 5-task set):

- `Microsoft\Windows\Application Experience\Microsoft Compatibility Appraiser`
- `Microsoft\Windows\Customer Experience Improvement Program` (whole folder)
- `Microsoft\Windows\Application Experience\ProgramDataUpdater`
- `Microsoft\Windows\Chkdsk\Proxy`
- `Microsoft\Windows\Windows Error Reporting\QueueReporting`

winutil (same 5, plus 6 more WU-specific, all recursive):

- `Microsoft\Windows\InstallService`
- `Microsoft\Windows\UpdateOrchestrator`
- `Microsoft\Windows\UpdateAssistant`
- `Microsoft\Windows\WaaSMedic`
- `Microsoft\Windows\WindowsUpdate`
- `Microsoft\WindowsUpdate`

2. Architecture comparison

Tool	Style	Notes
asl-win11	Modular — thin orchestrator (<code>asl-win11maker.ps1</code>) dot-sources a function library (<code>asl-win11.functions.ps1</code>) and calls a linear, annotated (<code>REQUIRED</code> / <code>OPTIONAL</code>) pipeline. Two arrays (<code>\$appPackagePrefixes</code> , <code>\$scheduledTaskPaths</code>) are the primary customization surface.	Easiest of the four to extend safely — new tweaks are a function + one annotated call.
ntdevlabs	Monolithic — single 535-line script, top to bottom, only two tiny helper functions (<code>Set-RegistryValue</code> / <code>Remove-RegistryValue</code>). Everything else is inline sequential code.	This is the upstream asl-win11 was forked and modularized from — explains the near-identical tweak set.
tiny11-automated (headless)	Monolithic — single 1003-line script, but organized as a flat set of ~30 <code>function</code> blocks followed by one top-level <code>try/catch</code> driver. No external modules; CI config (<code>config/matrix_config.json</code>) lives at the <i>workflow</i> level, not inside the script.	CI/CD-ready: fully parameterized (<code>-ISO</code> , <code>-INDEX</code> , <code>-SCRATCH</code> , <code>-SkipCleanup</code>), no interactive prompts, fails fast via <code>Test-Prerequisites</code> , emits <code>tiny11-buildinfo.json</code> for downstream workflow steps, has a Pester test (<code>Test-BuildNumberParsing.Tests.ps1</code>) covering its build-number parsing logic.
winutil	Plugin/integration point — a single function (<code>Invoke-WinUtilISOScript</code>) designed to be called from WinUtil's larger GUI toolkit. Takes an already-mounted image, a <code>-Log</code> scriptblock for caller-supplied logging, and switches like <code>-InjectCurrentSystemDrivers</code> / <code>-InstallEditionId</code> .	Not a standalone builder; it's the "customize a mounted image" step in someone else's pipeline. Cleanest interface for embedding, but owns none of the ISO lifecycle (copy/mount/export/burn).

3. Key gaps identified

What ntdevlabs has that asl-win11 lacks: Nothing feature-wise — asl-win11 is a modularized fork of ntdevlabs with an identical tweak/removal set, so there's no gap here. The only functional difference is architecture (modular vs. monolithic) and asl-win11's slightly larger app-removal list.

What tiny11-automated adds specifically for headless/CI use:

- Fully non-interactive parameter surface (`-ISO` , `-INDEX` , `-SCRATCH` , `-SkipCleanup`) — no `Read-Host` anywhere.
- `Resolve-ImageIndex` auto-corrects index drift instead of presenting a menu.
- `Test-Prerequisites` fails fast (non-admin, missing media, low disk space) instead of interactive recovery.
- Machine-readable `tiny11-buildinfo.json` output (`Write-BuildInfo`) for CI to consume build metadata.
- Top-level `try/catch` with emergency dismount/hive-unload and explicit `exit 0` / `exit 1` codes for CI pass/fail signaling.
- Dual console+file structured logging (`Write-Log` with levels) vs. plain transcript.
- A larger, more current app-removal list (57 prefixes, including AI/Recall/Copilot-era packages) and additional service disabling (`DiagTrack` , `WerSvc` , `PcaSvc` , `SysMain`).

Most comprehensive telemetry removal: tiny11-automated (headless) — same telemetry base as asl-win11/ntdevlabs, plus explicit service-level disabling (`DiagTrack` , `WerSvc` , `PcaSvc` , `SysMain` via `Start=4`) and dedicated Windows AI/Recall keys that don't exist in the other three.

Best edge-case handling (ESD conversion, recovery export): asl-win11, ntdevlabs, and tiny11-automated all implement the same core ESD→WIM conversion and `/Compress:recovery` export pass; tiny11-automated is the most *robust* of the three because of its fail-fast prerequisite checks and emergency-cleanup `try/catch` — a corrupted run is far less likely to leave a dangling image mount than the other two, which have only ad-hoc retry-on-cleanup logic. winutil doesn't participate in this category at all since it assumes a WIM is already prepared.

4. Recommended integrations for asl-win11 (prioritized by impact)

1. **Top-level error handling with emergency cleanup** (*High impact, low effort*) — Adopt tiny11-automated's pattern: `$ErrorActionPreference = 'Stop'` , a single top-level `try/catch` around the pipeline in `asl-win11maker.ps1` , and an emergency handler that discards any still-mounted images (`Get-WindowsImage -Mounted | Dismount-WindowsImage -Discard`) and unloads registry hives before exiting. Today a mid-pipeline failure can leave `install.wim`/`boot.wim` mounted and hives loaded, blocking the next run.
2. **autounattend.xml image-index injection** (*High impact, medium effort*) — Adopt winutil's `ConvertTo-WinUtilISOAnswerFile` approach: namespace-aware injection of `/IMAGE/INDEX` into the answer file's `<MetaData>` so Setup installs the edition actually selected via `Select-InstallImageIndex` , instead of relying on the answer file's own (possibly stale) assumptions. Directly fixes a correctness gap shared by all three PowerShell scripts.
3. **ei.cfg / PID.txt edition enforcement** (*Medium impact, low effort*) — Add winutil's `Write-WinUtilISOEditionConfig` -style step: write `sources\ei.cfg` pinning the selected edition and delete `sources\PID.txt` so Setup doesn't fall back to a stale firmware product key. Cheap, and prevents an edition-mismatch failure mode none of the three PowerShell scripts currently guard against.
4. **support\ folder removal from the ISO** (*Low impact, trivial effort*) — One-line size reduction, already proven safe by winutil.
5. **Additional WU-related scheduled task removals** (*Medium impact, trivial effort*) — Add winutil's 6 extra task paths (`InstallService` , `UpdateOrchestrator` , `UpdateAssistant` , `WaaSMedic` , `WindowsUpdate` x2) to `$scheduledTaskPaths` in `asl-win11maker.ps1` — pure additive config change, no new function needed.
6. **Structured/leveled logging + build-info JSON** (*Medium impact, medium effort*) — Borrow tiny11-automated's `Write-Log` (leveled, dual console+file) and `Write-BuildInfo` JSON emission. Useful if asl-win11 is ever driven from CI or wrapped by another tool; low risk since it's additive to the existing `Start-Transcript` call.

7. **Driver injection (`-InjectCurrentSystemDrivers -style`)** (*High impact for hardware compatibility, higher effort*) — Port winutil's `Export-WindowsDriver -Online` + DISM `/Add-Driver /Recurse` into both install.wim and boot.wim index 2. Valuable for building images targeted at specific hardware, but it's the largest new function to write/test of anything in this list — lower priority than the correctness/robustness items above unless there's an immediate need for it.
8. **Dual-path hardware-bypass strategy from Rufus (optional/defensive)** (*Low-medium impact, medium effort*) — Rufus's `wue.c` shows a fallback hierarchy worth knowing about even though asl-win11 already does direct offline-hive editing successfully: it *prefers* direct registry-hive mount/edit for the LabConfig bypass keys, and only falls back to embedding `reg add` commands in `RunSynchronousCommand` blocks inside `unattend.xml` when direct hive access isn't available (e.g., sandboxed/Store-distributed builds). Not urgent for asl-win11 today, but worth keeping in mind if this script is ever adapted to run in a more restricted environment. Rufus's other unattend.xml techniques worth knowing about even though they don't map to a direct ported feature: `BypassNRO` + `HideOnlineAccountScreens` for defeating the mandatory Microsoft-account OOBE screen (asl-win11 already does the `BypassNRO` half via `Enable-LocalAccountOOBE`), and the fact that Setup only auto-copies `Autounattend.xml` into `%WINDIR%\Panther\unattend.xml` when a windowsPE pass is present — a quirk worth remembering if asl-win11's autounattend.xml handling is ever reworked (item 2 above).

Not recommended for porting: setup-script pre-staging from `<sg:File>` extension nodes (winutil) — no current asl-win11 use case needs scripts injected via the answer file, and it adds XML-parsing complexity for a feature nothing in this pipeline currently calls for.

Appendix: original winutil vs. tiny11maker baseline comparison

(Retained as the original two-way analysis this document was built from; paths updated to reflect the current `asl-win11maker.ps1` / `asl-win11.functions.ps1` naming and the `reference/winutil/` submodule location.)

What winutil has that asl-win11 LACKS

1. Driver Injection (`-InjectCurrentSystemDrivers`)

- Exports all drivers from the running system via `Export-WindowsDriver -Online`
- Injects into install.wim via DISM `/Add-Driver /Recurse`
- Also injects into boot.wim index 2 (so setup PE has the same drivers)
- Temp directory created/cleaned up automatically
- **Verdict: Significant missing feature** — valuable for hardware compatibility

2. autounattend.xml XML Manipulation

- `ConvertTo-WinUtilISOAnswerFile` : namespace-aware XML processing that injects the `IMAGE/INDEX` metadata into the answer file so Windows Setup installs the correct edition
- Removes empty/invalid product keys from the XML before embedding
- `Get-WinUtilISOScriptChildElement` : helper for namespace-safe element creation
- **Verdict: asl-win11 just copies autounattend.xml as-is; it does not inject the image index into the XML**

3. Setup Script Pre-staging from autounattend.xml

- Parses <sg:File> extension nodes in autounattend.xml
- Writes each referenced script to its Windows path under the mount dir with correct encoding:
- .ps1 , .xml → UTF-8
- .reg , .vbs , .js → UTF-16 LE with BOM
- Ensures scripts survive Windows Setup stripping unrecognized XML namespaces
- **Verdict: Currently absent from asl-win11**

4. Edition Enforcement (-InstallEditionId)

- Write-WinUtilISOEditionConfig : writes sources\ei.cfg to pin the edition
- Deletes sources\PID.txt to prevent setup from forcing a stale firmware product key
- **Verdict: asl-win11 doesn't write ei.cfg or remove PID.txt**

5. ISO support\ Folder Removal

- Removes the support\ folder from the ISO contents directory
- Reduces final ISO size
- **Verdict: Not done in asl-win11**

6. Additional Scheduled Task Removals (6 more than asl-win11's 5)

winutil removes these task paths that asl-win11 skips:

- Microsoft\Windows\InstallService (recursive)
- Microsoft\Windows\UpdateOrchestrator (recursive)
- Microsoft\Windows\UpdateAssistant (recursive)
- Microsoft\Windows\WaaSMedic (recursive)
- Microsoft\Windows\WindowsUpdate (recursive)
- Microsoft\WindowsUpdate (recursive)
- **Verdict: More thorough WU task cleanup**

7. Additional Windows Update Registry Tweaks

winutil applies these that asl-win11 omits:

Registry Path	Value	Note
...WindowsUpdate\AU\AUOptions	1	Disable auto-update options

Registry Path	Value	Note
...WindowsUpdate\AU\UseWUServer	1	Force WSUS server policy
...WindowsUpdate\DisableWindowsUpdateAccess	1	Block WU access policy
WUServer	http://localhost:8080	More specific localhost URL
WUStatusServer	http://localhost:8080	Same
...UScheduler_Oobe\WindowsUpdate\workCompleted	1	Block WU OOBE install
...DeliveryOptimization\Config\DODownloadMode	0	Disable delivery optimization
Services\BITS\Start	4 (disabled)	Background Intelligent Transfer
Services\Usosvc\Start	4 (disabled)	Update Orchestrator Service
Services\WaaSMedicSvc\Start	4 (disabled)	WaaS Medic Service

asl-win11 instead **deletes** the WaaSMedicSvc and Usosvc keys entirely (more aggressive) and sets `DisableOnline = 1` on OOBE (which winutil doesn't do).

8. Pluggable Logger (`-Log` scriptblock parameter)

- winutil accepts a custom logging function
- Enables integration into larger workflows (e.g., GUI progress display in WinUtil itself)
- **Verdict: asl-win11 always writes to transcript/stdout**

What asl-win11 HAS that winutil LACKS

1. Full End-to-End ISO Build Workflow

winutil is a *processing step only* — it receives an already-mounted image path. asl-win11 owns the full pipeline:

- ESD → WIM conversion
- Interactive image index selection
- `Copy-SourceImageFiles` (ISO copy to scratch)
- `Mount-InstallImage` , `Show-ImageMetadata`
- `Complete-InstallImage` (DISM cleanup + recovery export)
- `Set-BootImageBypassTweaks` (boot.wim hardware bypass)
- `New-Tiny11Iso` (oscdimg ISO creation)
- `Invoke-Tiny11Cleanup` (eject, delete temps, retry logic)

- `Confirm-AutounattendXml` (download from GitHub if missing)

2. Edge Removal (File + Registry)

- `Remove-Edge` : removes Edge browser dirs, EdgeUpdate, EdgeCore
- Optional WebView2 removal (`-SkipWebView` switch)
- Cleans up Edge/EdgeUpdate uninstall registry entries
- **Not implemented in winutil**

3. OneDrive Setup Removal

- `Remove-OneDriveSetup` : removes `Windows\System32\OneDriveSetup.exe`
- Uses `takeown` / `icacis` for permission elevation
- **Not implemented in winutil**

4. Hardware Bypass in Boot.wim (`Set-BootImageBypassTweaks`)

- Mounts boot.wim index 2, loads hives, applies LabConfig bypass keys, saves
- winutil's driver injection touches boot.wim, but does **not** apply hardware bypass registry tweaks to it
- **Verdict: asl-win11 applies bypass to both install.wim and the setup PE**

5. Much Larger App Removal List (49 vs 19 packages)

asl-win11 removes 30+ additional package prefixes beyond winutil's 19, including:

- All Xbox app family (TCUI, App, GameOverlay, GamingOverlay, IdentityProvider, SpeechToText)
- Cortana (`Microsoft.549981C3F5F10`)
- Dolby OEM apps, Intel Management Status
- Mixed Reality Portal
- SkypeApp, People, Phone Link, WindowsCamera, WindowsAlarms, WindowsMaps
- Mail/Calendar (`windowscommunicationsapps`), OneNote UWP, OfficePushNotification
- Microsoft Family Safety, 3D Viewer, Wallet, Sticky Notes, ZuneVideo

winutil has `Microsoft.Paint` which asl-win11 currently has commented out.

6. DISM Image Cleanup Step

- `Complete-InstallImage` runs `/Cleanup-Image /StartComponentCleanup /ResetBase` before export
- Significantly reduces final image size
- **Not done in winutil (caller's responsibility)**

7. OOBE-Specific WU Suppression

- Sets `HKLM\zSOFTWARE\Microsoft\Windows\CurrentVersion\OOBE\DisableOnline = 1`
- Uses RunOnce commands to stop/disable WU service on first boot post-OOBE
- Fully deletes WaaSMedicSVC and UsSvc service keys (vs winutil setting Start=4)
- `DoNotConnectToWindowsUpdateInternetLocations = 1` policy key